MITTUNIVERSITETET
Laboration: L2 Linux
Laborant: Hussein Alshaikhshaker
Kurs: Operativ System (HT24-DT201G)
Datum: 2024-11-12

Kommandon och symboler

Starta terminalen [Ctrl + Alt + T]. Använd [Ctrl + C] för att avbryta exekveringen av kommandon vid behov, eller i värsta fall med en stoppsignal [Ctrl + Z].

1. ***Förklara kortfattat vad följande kommandon och program används för. Ge exempel som styrker eller förenklar din förklaring.***
   a. ***'ls'***
   b. ***'mkdir'***
   c. ***'mv'***
   d. ***'which'***
   e. ***'cat'***
   f. ***'chmod'***

a) 'ls' programmet listar information om filer och kataloger i nuvarande katalog. Data skrivs ut i alfabetsordning som standard. Utdata som börjar med '.' skrivs ut först. [1]
b) 'mkdir' skapar en ny katalog med ett användarangivet namn. Rapporterar felmeddelande om en katalog redan finns med samma namn. [1]
c) Syftet med 'mv' är att flytta filer/kataloger. Man kan även döpa om filer/katalogerna . Det finns tillval som [-i] som indikierar att kommandot lovar att inte skriva över filer som redan finns. [1]
d) 'which' används tillsammans med programnamn för att hitta sökvägen till filerna som körs när dessa program-kommandon används. [2]
e) 'cat' skriver ut innehållet av specifierad fil. Kommandot har många tillval, men en av användbara tillval är -n vilket visar radnummer för varje rad i filen. [2]
f) 'chmod' används för att ändra läsningsbehörighet, skrivningsbehörighet och exekveringsbehörighet åt grupper/användare för en fil/katalog. Dessa tre behörigheter betecknas som r, w och x(read, write och execute) vid användning av kommandot. Man kan även använda octal notation av fyra siffror vid ändring av behörigheter. [chmod g-rw EXMP] detta kommando raderar läs- och skrivbehörighet men inte exekvering för grupper. [4]

2. **Läs på om symbolerna | (pipe) samt > och < (redirect). Skapa därefter en rad kod som nyttjar båda koncepten med ett par kommandon för att lösa n˚agon enklare syssla. Förklara vad som uppnås.**

Pipes används för omdirigering av standardutdata till en destination. Pipes är enkelriktade. Data flödar från vänster till höger i pipen. Utdatat från första kommandot/processen blir då indata till nästa kommando osv.. [3]

Omdirigering [<<] & [>>] eller [<] & [>] används för att ”peka” på var kommandon ska arbeta sig mot. Det vill säga att med hjälp av dessa symboler kan man ange input och output mot olika kommandon. [3]

[ls | grep ”.txt” > text_files.txt]
Denna kommand kommer att använda programmet ”ls” för att lista textfiler. ”grep” filtrerar alla filer som innehåller ”.txt”. [>] indikerar här då att ls-kommandots indata blir utdata för text_files.txt. Om inte text_file.txt redan finns kommer den att skapa en ny textfil. Om jag använder [>>] istället så kommer den att behålla gamla innehållet och skriva nya innehållet. [2] [3]

Grundläggande systemdata

Studera följande kommandon och program: uname, df, free och vmstat.

3. Besvara följande frågor med hjälp av dessa kommandon tillsammans med kommandona från den föregående uppgiften. Ange för varje deluppgift vilket kommando med vilka tillval som använts för att nå resultatet. Kom ihåg enhet och prefix på applicerbara storheter.

a. Vilket operativsystem och vilken distribution körs?

b. Hur mycket fysiskt minne finns installerat?

c. Hur mycket växlingsutrymme finns tillgängligt? Kontrollera att värdet överensstämmer med filen swaps under /proc.

d. Hur mycket primärminne finns tillgängligt för nya processer?

e. Vilken sammanlagd kapacitet finns i de monterade filsystemen? Hur mycket av kapaciteten används? Ignorera nätverkslagring.

f. Vilken typ av filsystem används på den största lokala volymen?

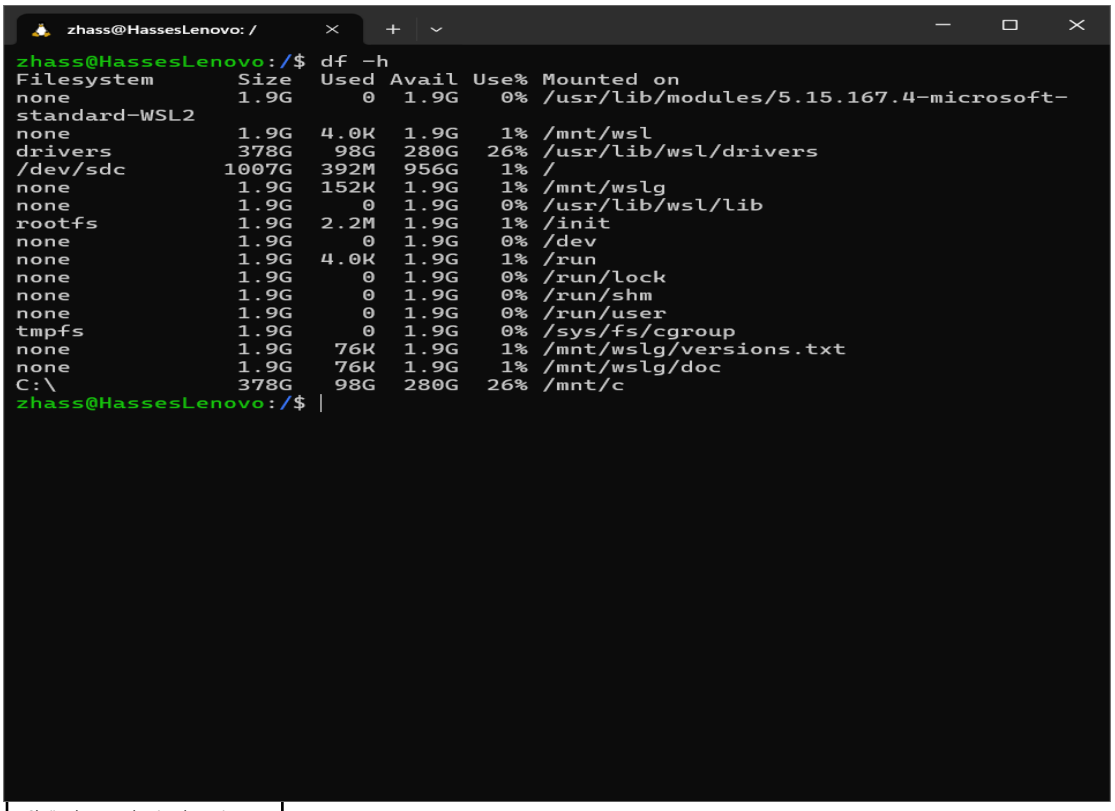
g. Var i filsystemet är programmen uname, df, free och vmstat lagrade?

h. Vilken processormodell används? Besvara även hur mycket cacheminne och vilken klockfrekvensprocessorn arbetar med. Du hittar informationen i filen cpuinfo under /proc. Om flera processorer listas, utgå ifrån den första.

a) Linux 5.15.167.4-microsoft-standard-WSL2 //[uname -a]
Distribution: "Debian GNU/Linux" //[cat etc/os-release]
b) 4034MB //[free –-mega]
c) 1073MB //[free –-mega]
d) 3567 MB //[free –-mega]
e) 1407.8GB , Använt : 98.394548 GB //[df -h]
f) ext4 //[df -h]
g) Dom finns alla i samma fil. /usr/bin/ //[which med kommandot som argument]
h) Modell: 154 //[cat /proc/cpuinfo]
CPU-Klockfrekvens: 3495.999MHz
Cache-storlek: 12288KB



Skrämdump av cpu-info.



Skrämdump av lagringskapacitet.



Skrämdump av fysiskt minne.

Prestanda och processer

Två användbara verktyg för realtidsövervakning ä top och den mer avancerade htop. De kan närmast likställas aktivitetshanteraren i Windows. Till skillnad från många andra program i Linux är top och htop omfattande. Tryck "h" under körning för att se olika kortkommandon som stöds och "q" för quit. Starta top

4.

a) Hur många processer finns i respektive tillstånd (eng. state)? Hur många processer finns totalt?

b) Förklara detaljerat vad de olika tillstånden innebär. Endast tillstånden listade i tops sammanställning behöver behandlas.

c) Vilken process använder mest minne?

d) Vad är högsta respektive lägsta värdet för PR?

e) Vad är innebörden av det kolumnerna PR och NI visar?

f) Förklara det matematiska förhållandet mellan PR och NI. Vilka exempel på förhållandet kan du se i htop?

g) Starta gedit, firefox eller något annat vanligt användarprogrammet. Försök att sänka prioriteten på processen så mycket som möjligt genom att använda renice. Försök även att höja prioriteten så mycket som möjligt. Vad blir ditt resultat? Var går gränserna? Kom ihåg att visa använda kommandon.

h) Se till att firefox är helt avstängt inför denna uppgift. Starta nu firefox genom terminalen. Gå till www.youtube.com och starta valfri video. Skicka nu [Ctrl + Z] till processen genom terminalen. Försök därefter att skapa ett par nya flikar i firefox, endera genom att klicka på plus eller med genvägen [Ctrl + T] i firefox. Vad händer med processen? Vad säger htop? Observera bland annat tillståndet. Kör kommandot fg i samma terminal när du är klar med uppgiften. Vad händer?

I) Stäng terminalen som du använde för att starta firefox i föregående uppgift. Vad händer? Varför?

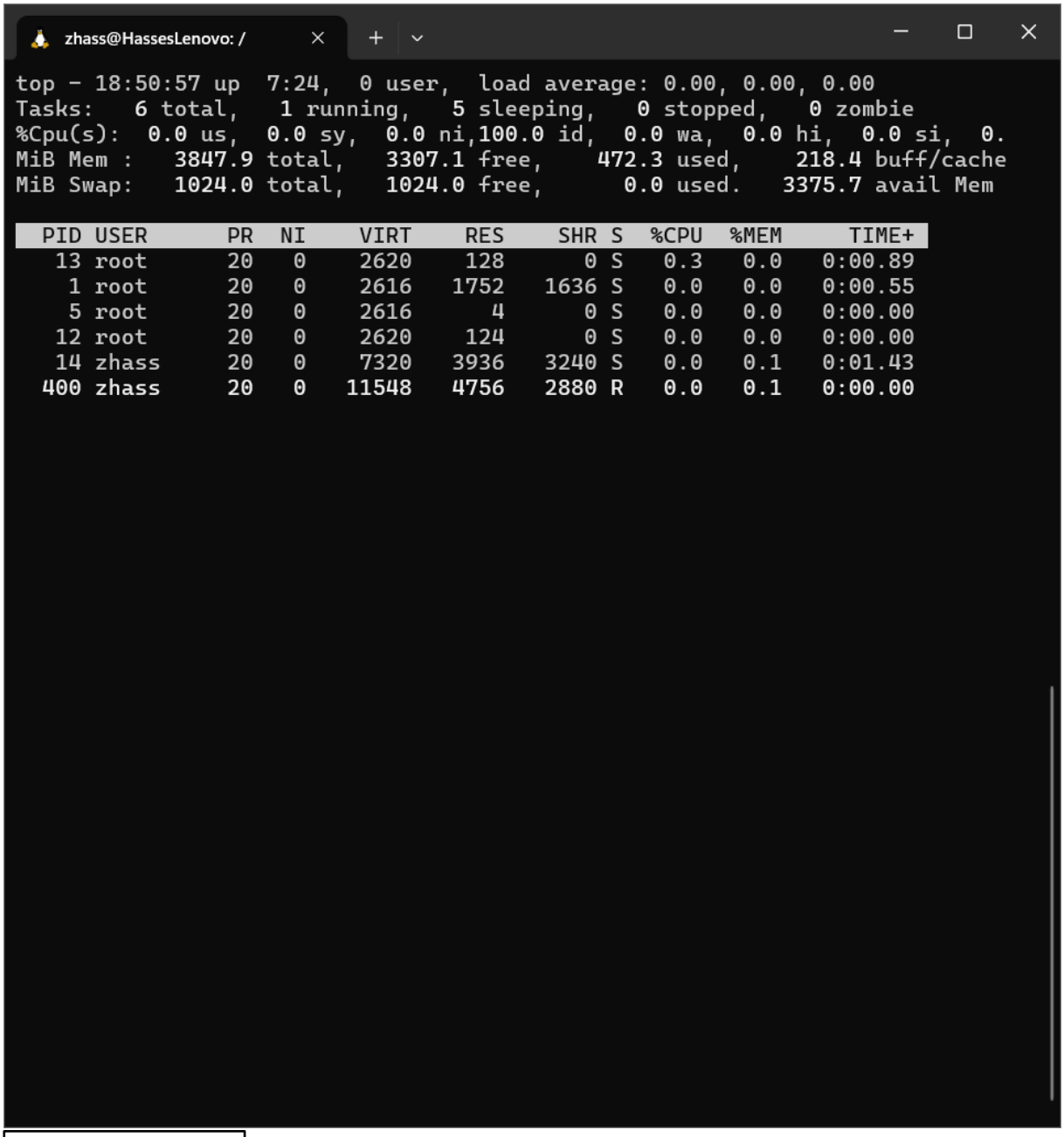
a) Totalt sex processer. En kör, fem sleeping, noll stopped, noll zombies.
b) Tasks består av räknare som räknar processernas tillstånd. Running räknar hur många processer som exekveras just nu eller väntar på processorn så att den får resurser och börjar exekvera den processen. Sleeping innebär att ett program/process exekveras för tillfället men väntar på händelse eller interaktion. Stopped innebär att en process/program har blivit stoppad under exekvering. Den är inte helt avslutad och kan återupptas vid signal från användare. Tillståndet zombie sker när en process/program har exekverat klart och väntar på att föräldern ska ta bort den från minnet. [4] [5]
c) Bash och htop. Båda använder 0.1% av primärminnet.
d) Alla processer har PR 20 och NI värdet 0.
e) Schemaläggaren är ansvarig för vilka processer som ska exekveras härnäst. Det är inte möjligt att påverka själva schemaläggare direkt, men man kan låta den veta vilka processer användaren prioriterar och schemaläggaren kommer att ta hänsyn till det. Niceness(NI) kan ändras av användare. Värdet kan gå från -20, vilket är det högsta värdet för ett program/process till 19 som är då det lägsta. Lägre värde = högre prio.
   Priority(PR) är kärn-baserat. Dvs det kan inte ändras av användare, men samtidigt är den beroende av NI värdet. PR värden kan utsträcka sig från 0 till 139. Vanligtvis 0-99 för real-time och 100-139 för användare. [6]
f) PR = 20 + NI. By default har alla användarprocesser NI värdet 0 om man inte har ändrat. Så PR får automatiskt värdet 20 som standard. Beroende på vilka NI

värden man angett kommer schemaläggaren att prioritera olika processer och ge dem mer CPU-tid. [6]

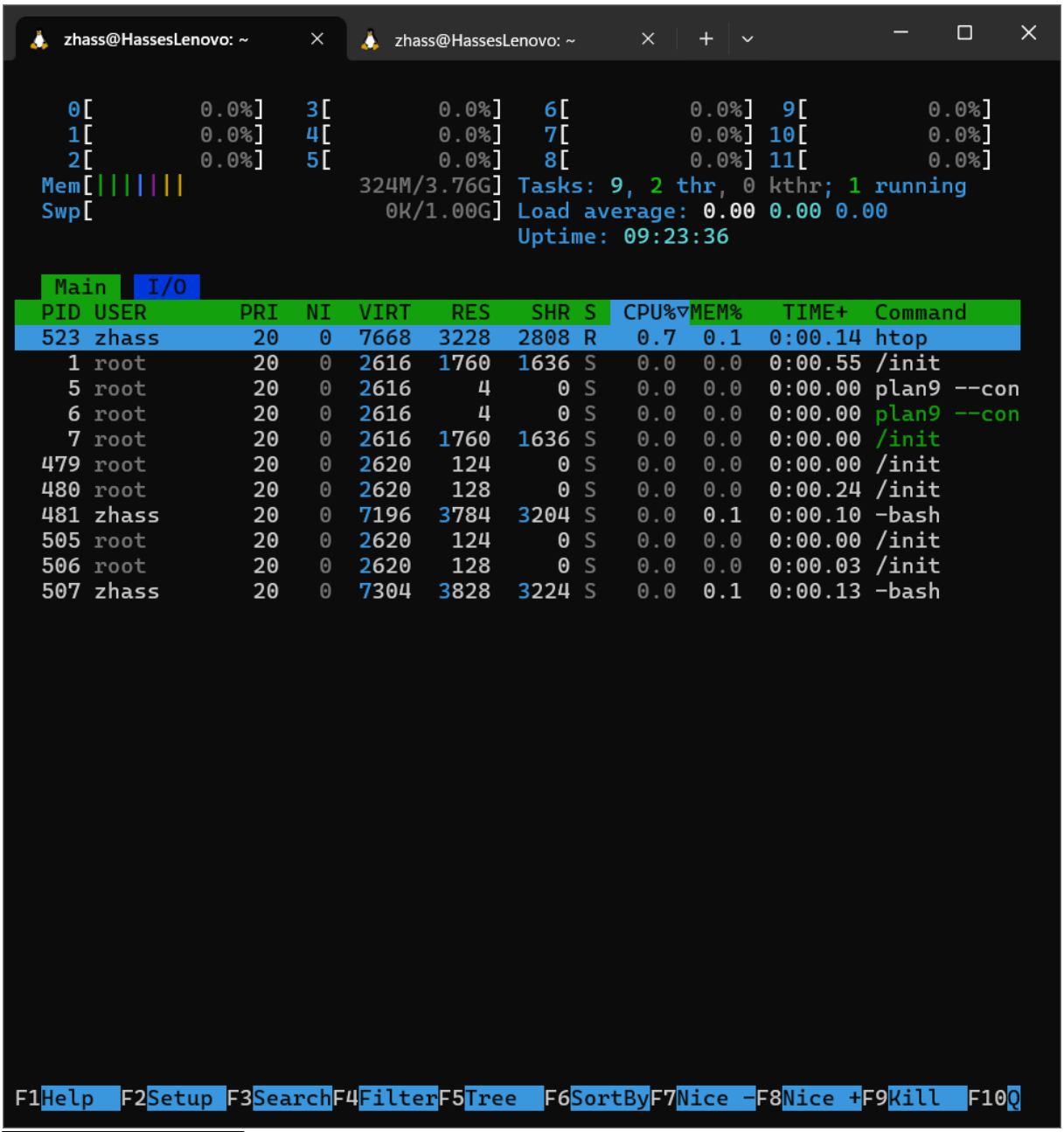
g) Jag försökte höja prioriteten för processen htop genom att använda [sudo renice x -p x]. Det lägsta NI värdet tillåtet var -20 vilket resulterade att PR värdet fick värdet 0(högst prioriterat). För att ge det mindre prioritet var högsta NI värdet 19, vilket satte PR värdet till 39.

h) Det gick inte att öppna flera fönster efter att ha utfört Ctrl-Z pga att processen var stoppad enligt htop. Sedan utfördes [fg] i samma terminal som firefox och då processen fortsatte att exekveras precis där den stoppades.

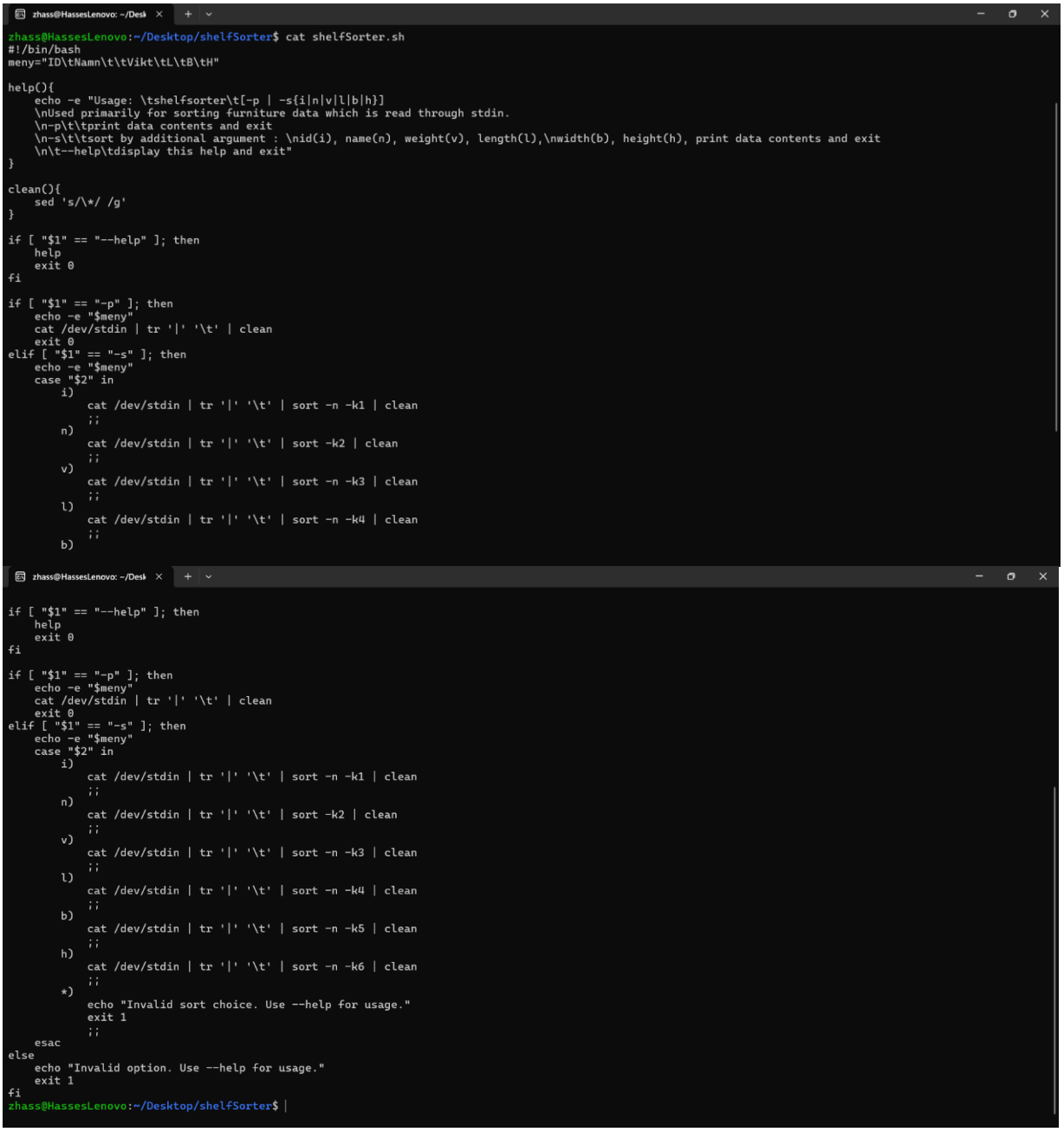
i) Firefox stängdes pga att jag stängde ned terminalen som är förälderprocessen för firefox. Barnprocesser är beroende av förläderprocesser, det resulterar i att förälderprocesser ”dödar sina barn” genom att skicka en signal till dess barnprocesser.



Skrämdump av programmet top.



Skrämdump av programmet htop.



```
zhass@HassesLenovo:~/Desktop/shelfSorter$ cat shelfSorter.sh
#!/bin/bash
meny="ID\tNamn\t\tVikt\tL\tB\tH"

help(){
    echo -e "Usage: \tshelfsorter\t[-p | -s{i|n|v|l|b|h}]
    \nUsed primarily for sorting furniture data which is read through stdin.
    \n-p\t\tprint data contents and exit
    \n-s\t\tsort by additional argument : \nid(i), name(n), weight(v), length(l),\nwidth(b), height(h), print data contents and exit
    \n\t--help\tdisplay this help and exit"
}

clean(){
    sed 's/\*/ /g'
}

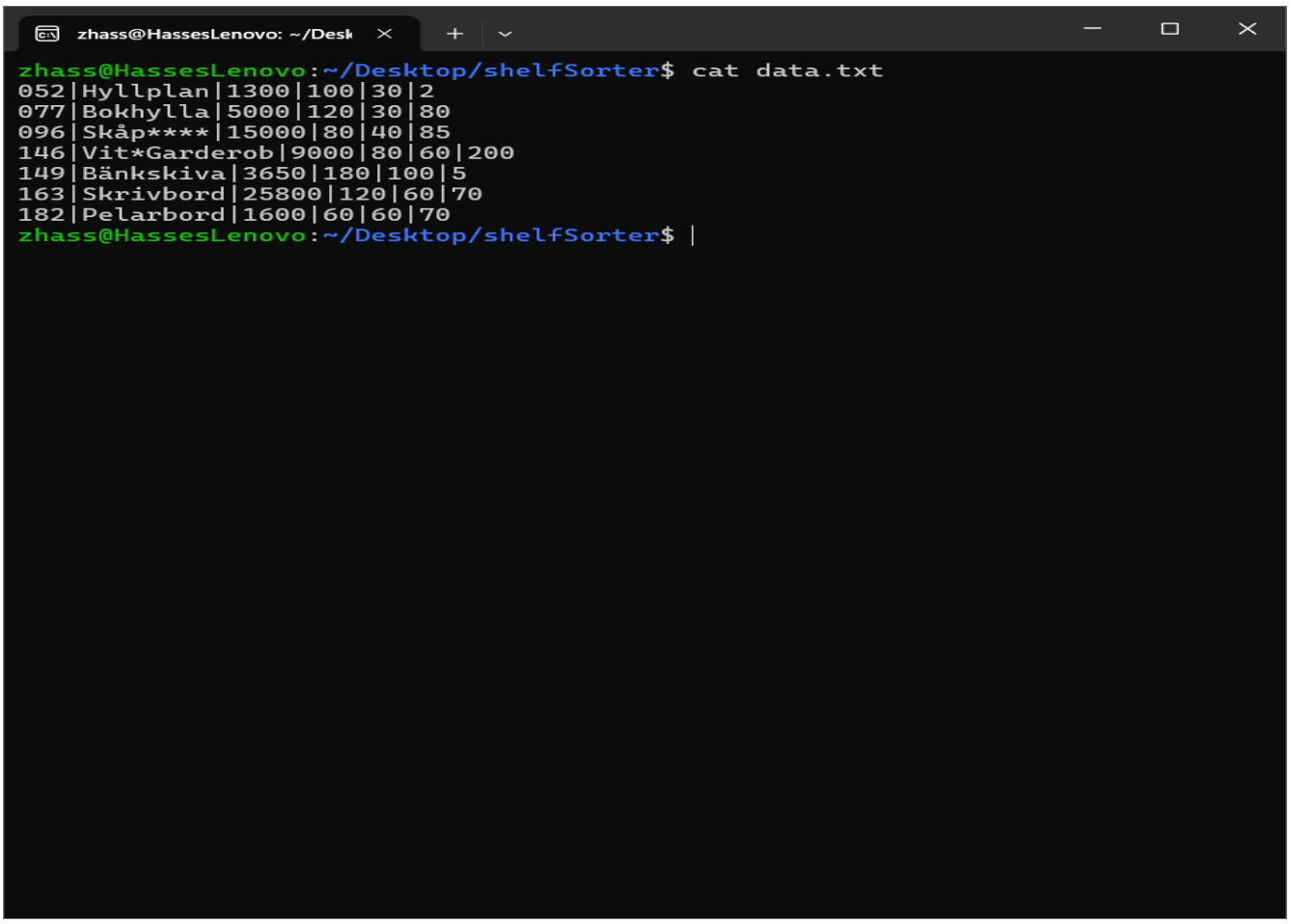
if [ "$1" == "--help" ]; then
    help
    exit 0
fi

if [ "$1" == "-p" ]; then
    echo -e "$meny"
    cat /dev/stdin | tr '|' '\t' | clean
    exit 0
elif [ "$1" == "-s" ]; then
    echo -e "$meny"
    case "$2" in
        i)
            cat /dev/stdin | tr '|' '\t' | sort -n -k1 | clean
            ;;
        n)
            cat /dev/stdin | tr '|' '\t' | sort -k2 | clean
            ;;
        v)
            cat /dev/stdin | tr '|' '\t' | sort -n -k3 | clean
            ;;
        l)
            cat /dev/stdin | tr '|' '\t' | sort -n -k4 | clean
            ;;
        b)
```

```
if [ "$1" == "--help" ]; then
    help
    exit 0
fi

if [ "$1" == "-p" ]; then
    echo -e "$meny"
    cat /dev/stdin | tr '|' '\t' | clean
    exit 0
elif [ "$1" == "-s" ]; then
    echo -e "$meny"
    case "$2" in
        i)
            cat /dev/stdin | tr '|' '\t' | sort -n -k1 | clean
            ;;
        n)
            cat /dev/stdin | tr '|' '\t' | sort -k2 | clean
            ;;
        v)
            cat /dev/stdin | tr '|' '\t' | sort -n -k3 | clean
            ;;
        l)
            cat /dev/stdin | tr '|' '\t' | sort -n -k4 | clean
            ;;
        b)
            cat /dev/stdin | tr '|' '\t' | sort -n -k5 | clean
            ;;
        h)
            cat /dev/stdin | tr '|' '\t' | sort -n -k6 | clean
            ;;
        *)
            echo "Invalid sort choice. Use --help for usage."
            exit 1
            ;;
    esac
else
    echo "Invalid option. Use --help for usage."
    exit 1
fi
zhass@HassesLenovo:~/Desktop/shelfSorter$
```

Skrämdump av skalskriptet



Skrämdump av textfilen

Källhänvisningar

[1] *"Info -command". [Inbyygt program i Linux]*

[2] ”freeCodeCamp,” [Online]. Available: https://www.freecodecamp.org/news/linux-terminal-piping-and-redirection-guide/.

[3] ”Föreläsning,” [Online]. Available: https://apachepersonal.miun.se/~jimahl/rec/DT201G/3_processer.mp4

[4] ”IBM,” [Online]. Available: https://www.ibm.com/docs/en/aix/7.2?topic=commands

[5] [baeldung]. Available: https://www.baeldung.com/linux/process-states

[6] [peteris]. Available: https://peteris.rocks/blog/htop